

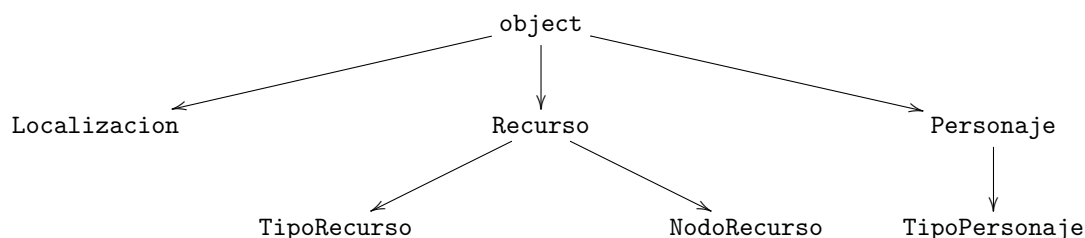
Práctica 3: Representación de dominios y resolución de problemas con técnicas de planificación

Autor: Jesús Muñoz Velasco

1. Ejercicio 1

1.1 Diseño del dominio

El dominio **tierra-media** del Ejercicio 1 establece la base sobre la que se construyen todos los ejercicios posteriores. Se establece la siguiente jerarquía de tipos:



donde

- **Localizacion** representa lugares del mapa (Hobbiton, Bree, Rivendell, etc.). Se utilizan principalmente para especificar los caminos que existen (teniendo que declararse bidireccionalmente con el predicado **camino** que se describirá más adelante)
- **TipoRecurso** representa un tipo de recurso extraíble (Mineral, Mithril, Madera, etc.). Se utiliza para extracciones y construcciones.
- **NodoRecurso** representa un nodo/yacimiento de un recurso en una localización concreta. Sirve principalmente para especificar la localización de un recurso (con el predicado **nodoEn**).
- **Recurso** serviría para especificar una instancia concreta de un recurso pero en este problema en concreto no es necesario. En su lugar se utiliza por pura semántica para agrupar **TipoRecurso** y **NodoRecurso** (ya que semánticamente están relacionados como atributos de un recurso).
- **Personaje** representa instancias concretas de personajes (Enano1, Hobbit1, etc.).
- **TipoPersonaje** representa categoría de personaje (Enano, Hobbit, etc.)

Se hace uso de constantes para fijar los tipos de personaje (**Enano**, **Hobbit**) y los cinco tipos de recurso (**Mineral**, **Mithril**, **Madera**, **Especia**, **Alimento**), en lugar de declararlos como objetos, lo que permite referenciarlos directamente en las precondiciones sin necesidad de instanciarlos en cada problema (aplicando el principio de desacople del problema y el dominio).

Se emplean además los siguientes predicados:

- (**estaEn** [**Personaje**] [**Localizacion**]) registra la posición de cada personaje.
- (**nodoEn** [**NodoRecurso**] [**Localizacion**]) registra la localización de un recurso.
- (**tipoNodo** [**NodoRecurso**] [**TipoRecurso**]) especifica qué tipo de recurso se encuentra en **NodoRecurso**.
- (**camino** [**Localizacion**] [**Localizacion**]) sirve para representar el mapa, señalando que existe un camino desde la primera localización hasta la segunda (habrá que usar dos veces este predicado si el camino es bidireccional pero cambiando origen por destino).
- (**caminoDestruible** [**Localizacion**] [**Localizacion**]) indica que al atravesar el camino desde la primera localización hasta la segunda este se destruirá (en ambos sentidos). Habrá que usarlo dualmente de la misma manera que el predicado anterior para que se destruya independientemente del sentido en el que se recorra dicho camino. En nuestro problema solo se usa para el camino Rivendell↔ Moria.
- (**disponible** [**Personaje**]) señala si un personaje no está asignado a ninguna tarea.

- (`trabajando [Personaje] [Localizacion] [TipoRecurso]`) indica que el personaje está trabajando en una cierta localización extrayendo un cierto recurso del tipo indicado.
- (`puedeExtraer [TipoPersonaje] [TipoRecurso]`) registra que el personaje de dicho tipo puede extraer recursos del tipo `TipoRecurso` correspondiente.
- (`esDeTipo [Personaje] [TipoPersonaje]`) sirve para relacionar cada instancia de personaje con su tipo (por ejemplo `Humano1` es de tipo `Humano`).

Además, de acuerdo al enunciado se definen las siguientes acciones¹:

Acción	Precondiciones	Efectos
Viajar ?p ?l1 ?l2	- ?p está en ?l1. - Existe un camino entre ?l1 y ?l2. - ?p está disponible.	- ?p está ahora en ?l2. - ?p ya no está en ?l1. - Se elimina el camino si procede
ExtraerRecurso ?p ?l ?tr	- ?p está en ?l. - ?p está disponible. - Existe un nodo del tipo ?tr en ?l. - El tipo de personaje de ?p tiene capacidad para extraer ese recurso.	- ?p se marca como trabajando en ?l extrayendo ?tr. - ?p deja de estar disponible.

Tabla 1: Acciones del dominio 1

1.2 Diseño del problema

Para el diseño del problema se declaran en primer lugar los objetos correspondientes a

- Localizaciones en el mapa.
- Personajes (instancias).
- Nodos de recursos.

según el enunciado (para más información se puede leer el archivo `problema1.pddl`).

En el estado inicial se definen los siguientes aspectos (también de acuerdo al enunciado):

- Posición inicial de cada personaje (de cada instancia).
- Disponibilidad de cada personaje (de cada instancia).
- Tipos de personaje (de cada instancia).
- Capacidad de extracción de cada personaje (de cada tipo de personaje).
- Ubicación de los nodos de recursos.
- Tipo de recurso de cada nodo.
- Mapa de caminos. Se declaran bidireccionalmente para indicar que se puede transitar en ambos sentidos (ya que el enunciado no indica lo contrario).

Finalmente, como objetivo se especifica que se verifiquen las siguientes condiciones:

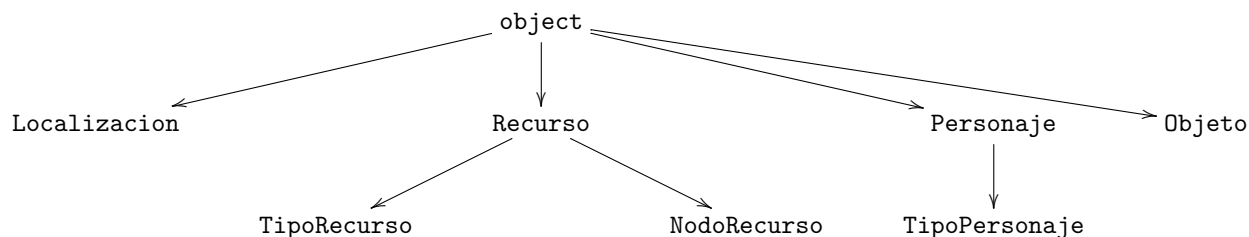
- Que haya un personaje asignado a un nodo de Mithril (en este caso solo puede ser un enano, y como solo hay uno tendrá que ser necesariamente `Enano1`).
- Que haya un personaje asignado a un nodo de Alimento (de nuevo solo hay una posibilidad y es que el `Hobbit1` esté extrayendo alimento en `Hobbiton`).

¹La notación empleada será: ?p para personajes, ?l para localizaciones y ?tr para tipos de recurso. Si hay varios del mismo tipo se usará un índice tras la letra.

2. Ejercicio 2

2.1 Diseño del dominio

El dominio del Ejercicio 2 extiende el del Ejercicio 1 con los elementos necesarios para modelar la Comunidad del Anillo y la destrucción del Anillo Único. La jerarquía de tipos se amplía con un nuevo tipo raíz:



donde **Objeto** representa ítems físicos portables por los personajes (Anillo, ChalecoMithril y Espada).

En las constantes se añade el tipo de personaje **Mago** y los tres objetos especiales del dominio (**Anillo**, **ChalecoMithril**, **Espada**), que se declaran directamente como constantes de tipo **Objeto** por las mismas razones de desacople enumeradas en el Ejercicio 1.

Se incorporan los siguientes predicados nuevos:

- (**comunidadFormada**) predicado sin parámetros que se activa cuando se ejecuta la acción **formarComunidad**. A partir de ese momento los miembros de la Comunidad deben desplazarse conjuntamente.
- (**esMiembro** [**Personaje**]) indica que el personaje pertenece a la Comunidad del Anillo.
- (**tieneObjeto** [**Personaje**] [**Objeto**]) registra que un personaje porta actualmente un objeto.
- (**objetoEn** [**Objeto**] [**Localizacion**]) indica que un objeto se encuentra en una localización (aún sin recoger).
- (**portadorAnillo** [**Personaje**]) señala que el personaje es el portador del Anillo Único.
- (**anilloDestruido**) predicado sin parámetros que se activa cuando el Anillo es destruido.
- (**lugarDestruccion** [**Localizacion**]) marca la localización donde debe destruirse el Anillo.
- (**chalecoMaterializado**) predicado sin parámetros que evita que el Chaleco de Mithril pueda materializarse más de una vez.

Respecto a las acciones, **Viajar** se modifica y se añaden cinco acciones nuevas:

Acción	Precondiciones	Efectos
Viajar ?p ?l1 ?l2 (modificada)	- Igual que en el Ejercicio 1. - Adicionalmente: si la comunidad está formada, ?p no puede ser miembro (los miembros deben usar <code>viajarComunidad</code>).	- Igual que en el Ejercicio 1.
formarComunidad ?p1 ?p2 ?l	- La comunidad no está formada aún. - ?p1 y ?p2 están disponibles y en la misma localización ?l. - ?p1 es de tipo Hobbit y ?p2 es de tipo Mago.	- Se activa <code>comunidadFormada</code>. - ?p1 y ?p2 pasan a ser miembros.
viajarComunidad ?p1 ?p2 ?org ?dst	- La comunidad está formada. - ?p1 y ?p2 son miembros, están disponibles y en ?org. - Existe camino entre ?org y ?dst.	- Ambos se mueven a ?dst. - Se elimina el camino si procede.
materializarChaleco ?mago ?l	- La comunidad está formada. - ?mago es de tipo Mago, es miembro, está disponible y en ?l. - Hay algún personaje extrayendo Mithril en ?l. - El chaleco no ha sido materializado todavía.	- ChalecoMithril aparece en ?l. - Se activa <code>chalecoMaterializado</code>.
recogerObjeto ?p ?l ?o	- La comunidad está formada y ?p es miembro, está disponible y en ?l. - El objeto ?o está en ?l. - Si ?o es el Anillo: ?p debe ser Hobbit. - Si ?o no es el Anillo: ?p debe ser ya <code>portadorAnillo</code> (garantiza que el Anillo se recoja primero).	- ?p pasa a portar ?o. - ?o deja de estar en la localización. - Si ?o es el Anillo, ?p se convierte en <code>portadorAnillo</code>.
destruirAnillo ?p ?l	- ?p es <code>portadorAnillo</code> y porta los tres objetos (Anillo, ChalecoMithril, Espada). - ?p está en ?l y ?l es el <code>lugarDestruccion</code>. - El Anillo no ha sido destruido todavía.	- Se activa <code>anilloDestruido</code>. - ?p deja de ser <code>portadorAnillo</code> y de portar el Anillo.

Tabla 2: Acciones nuevas/modificadas del dominio 2

2.2 Diseño del problema

El mapa de localizaciones y los nodos de recursos son idénticos a los del Ejercicio 1. Los cambios en los objetos declarados son:

- Se sustituyen los dos enanos y el hobbit del ejercicio anterior por una plantilla de personajes más amplia: cuatro Hobbits (`Hobbit1`–`Hobbit3` en `Hobbiton`, `Hobbit4` en `Bree`), dos Magos (`Mago1` en `Rivendell`, `Mago2` en `Isengard`) y dos Enanos (`Enano1` en `Moria`, `Enano2` en `Isengard`).

En el estado inicial se añaden o modifican los siguientes aspectos:

- `Enano2` comienza *no disponible* (a diferencia del Ejercicio 1, donde todos los personajes iniciales estaban disponibles).
- Se declaran las capacidades de extracción del nuevo tipo `Mago` (en este problema ninguna, de modo que no aparece ningún predicado `puedeExtraer Mago ...`).
- Se especifica la ubicación inicial de los dos objetos recogibles en el mundo: el `Anillo` en `Rivendell` y la `Espada` en `Lothlorien`. El `ChalecoMithril` no tiene posición inicial porque se genera dinámicamente mediante la acción `materializarChaleco`.
- Se declara (`lugarDestruccion Orodruin`) como el único lugar válido para destruir el Anillo.

Finalmente, el objetivo se simplifica a un único predicado:

- (**anilloDestruido**): el plan tendrá éxito cuando el Anillo haya sido destruido en Orodruin. Esto implica implícitamente que se haya formado la Comunidad, recogido los tres objetos y llegado al lugar de destrucción.

3. Ejercicio 3

3.1 Diseño del dominio

El Ejercicio 3 extiende el dominio del Ejercicio 2 con tres bloques de cambios: un nuevo predicado de posición colectiva, dos predicados auxiliares de rendimiento y un conjunto de optimizaciones trasversales aplicadas en las acciones.

En cuanto a los predicados nuevos:

- (**comunidadEstaEn** [Localizacion]) registra la posición colectiva de la Comunidad como una única variable de estado. Este predicado sustituye funcionalmente a los predicados **estaEn** individuales de los miembros una vez formada la Comunidad: al ejecutar **formarComunidad**, los predicados **estaEn** de todos los miembros se retiran del estado y se inicializa **comunidadEstaEn**. A partir de ese momento, las acciones **viajarComunidad**, **materializarChaleco**, **recogerObjeto** y **destruirAnillo** comprueban exclusivamente **comunidadEstaEn**, nunca **estaEn**. Esto reduce significativamente el espacio de estados del planificador, pues N flujos de posición individuales quedan colapsados en uno solo.
- (**hayNodoTipo** [TipoRecurso] [Localizacion]) predicado auxiliar de rendimiento precalculado en el archivo de problema. Certifica que existe al menos un nodo del tipo de recurso indicado en la localización dada, eliminando el cuantificador **exists** sobre **NodoRecurso** que aparecía en la precondition de **ExtraerRecurso** en el Ejercicio 2. Su eliminación reduce el coste de la traducción a SAS+ y el tamaño del espacio de búsqueda.
- (**puedeExtraerPersonaje** [Personaje] [TipoRecurso]) predicado auxiliar de rendimiento también precalculado en el problema. Despliega explícitamente todas las combinaciones (personaje, recurso) válidas, eliminando el **exists** sobre **TipoPersonaje** que también existía en **ExtraerRecurso**. Al tratarse de la acción con mayor número de instanciaciones en el *grounding*, este predicado tiene un impacto directo sobre el tiempo de planificación.
- (**esMenor** [Personaje] [Personaje]) predicado auxiliar de ruptura de simetrías. Impone un orden total sobre los Hobbits de la Comunidad (declarado en el problema), lo que permite al planificador descartar permutaciones equivalentes en las acciones **formarComunidad** y **viajarComunidad**, reduciendo drásticamente el factor de ramificación cuando hay varios Hobbits con el mismo tipo en la comunidad. Este predicado solo está presente a partir del apartado (b) ya que es cuando aparecen varios Hobbits.

Las modificaciones sobre las acciones existentes son las siguientes:

- **ExtraerRecurso** reemplaza los dos **exists** originales por los predicados auxiliares **hayNodoTipo** y **puedeExtraerPersonaje**.
- **formarComunidad**, además de marcar la comunidad y sus miembros, ahora retira los predicados **estaEn** de todos los miembros y establece **comunidadEstaEn**, lo que descarta su posición individual.
- **viajarComunidad**, **materializarChaleco**, **recogerObjeto** y **destruirAnillo** pasan a usar **comunidadEstaEn** en lugar de **estaEn** para verificar la localización de la Comunidad.
- **viajarComunidad** añade restricciones de tipo sobre sus parámetros personaje (e.g. **esDeTipo** ?p1 Hobbit) y, donde procede, condiciones **esMenor** entre los Hobbits, con el doble propósito de acotar el *grounding* y eliminar simetrías.

3.2 Diseño del problema

Los objetos, el mapa de caminos, los nodos de recursos, la ubicación de los objetos especiales y el objetivo son idénticos en las cuatro variantes. Las diferencias entre apartados son exclusivamente la composición de la Comunidad, la presencia o ausencia de las relaciones **esMenor**, y los valores **esDeTipo** adicionales. En todos los casos el conjunto de personajes es el mismo: **Hobbit1–Hobbit3** en Hobbiton, **Hobbit4** en Bree, **Mago1** en Rivendell, **Mago2** en Isengard, **Elfo1** en Lothlorien, **Enano1** en Moria y **Enano2** en Fangorn, todos disponibles. Las diferencias por variante se detallan en las subsecciones siguientes.

3.3 Variante (a): 1 Hobbit + 1 Mago

La comunidad la forman exactamente un Hobbit y un Mago. La acción **formarComunidad** acepta dos parámetros de personaje (?p1 Hobbit, ?p2 Mago) y **viajarComunidad** también lleva exactamente esos dos. Al haber un solo Hobbit en la comunidad no existe simetría entre Hobbits que romper, por lo que el predicado **esMenor** no se introduce ni en el dominio ni en el problema.

3.4 Variante (b): 2 Hobbits + 1 Mago

Se amplía la comunidad a tres miembros: dos Hobbits y un Mago. Esto introduce la primera simetría real entre personajes del mismo tipo, ya que los dos Hobbits son intercambiables en la asignación de parámetros. Para gestionarla se añade el predicado **esMenor** y, en el problema, se declara el orden **Hobbit1 < Hobbit2 < Hobbit3 < Hobbit4**. Tanto **formarComunidad** como **viajarComunidad** exigen (**esMenor** ?p1 ?p2) sobre los dos Hobbits, reduciendo a la mitad las instanciaciones equivalentes en esas acciones.

3.5 Variante (c): 3 Hobbits + 1 Mago

La comunidad pasa a cuatro miembros: tres Hobbits y un Mago. Las simetrías entre los tres Hobbits crecen a $3! = 6$ permutaciones. Se aplica ruptura de orden total entre los tres Hobbits mediante las condiciones (**esMenor** ?p1 ?p2), (**esMenor** ?p1 ?p3) y (**esMenor** ?p2 ?p3), tanto en **formarComunidad** como en **viajarComunidad**, reduciendo el número de instancias válidas de 6 a 1 por combinación de Hobbits.

3.6 Variante (d): 3 Hobbits + 1 Mago + 1 Elfo

Se incorpora un quinto miembro de tipo **Elfo**. Las constantes del dominio ya incluían **Elfo** como tipo de personaje desde los ejercicios anteriores, pero es aquí donde se instancia por primera vez en la comunidad. La ruptura de simetrías entre los tres Hobbits se mantiene exactamente igual que en (c). La acción **viajarComunidad** gana un quinto parámetro de personaje con la restricción (**esDeTipo** ?p5 Elfo), y **formarComunidad** exige igualmente un quinto miembro de tipo **Elfo**.

3.7 Análisis de resultados

Tamaño de la comunidad	Tiempo de planificación (<i>Planner Time</i>)	Longitud del plan encontrado (<i>Plan Length</i>)
(a): 1 hobbit y 1 mago	0.34s	14
(b): 2 hobbits y 1 mago	0.45s	16
(c): 3 hobbits y 1 mago	0.45s	17
(d): 3 hobbits, 1 mago y 1 elfo	1267.89s	23

Tabla 3: Resultados de ejecución del Ejercicio 3

Los resultados revelan tres patrones de interés. En primer lugar, la transición de (a) a (b) y de (b) a (c) produce incrementos moderados tanto en tiempo como en longitud del plan. En tiempo, el salto de (a) a (b) se debe fundamentalmente al aumento del factor de ramificación de **formarComunidad** y **viajarComunidad** al considerar un Hobbit adicional; la práctica ausencia de crecimiento entre (b) y (c) demuestra que la ruptura de simetrías mediante **esMenor** absorbe casi por completo el coste teórico de multiplicar por tres las permutaciones de Hobbits. En longitud del plan, el incremento de una o dos acciones entre variantes refleja los pasos adicionales necesarios para reunir en un mismo punto a un miembro más de la comunidad antes de poder formarla.

En segundo lugar, el salto de 0.45s a más de 1267s en el apartado (d) no se explica únicamente por añadir un miembro más, sino por la introducción del tipo **Elfo**. A diferencia de los Hobbits adicionales de (b) y (c), cuyas simetrías se gestionan con **esMenor**, el Elfo es un tipo distinto sin miembro del mismo tipo en la comunidad y con posición inicial en Lothlorien, lo que obliga al planificador a explorar rutas de reunificación radicalmente nuevas. La combinación de un espacio de búsqueda más grande (cinco parámetros en **viajarComunidad** frente a cuatro) con la ausencia de restricciones adicionales de tipo entre los miembros no Hobbit da lugar a un crecimiento exponencial del tiempo de planificación.

Finalmente, el aumento de la longitud del plan de 17 a 23 acciones en el apartado (d) está en línea con la necesidad de desplazamientos adicionales para reunir al Elfo, que parte de Lothlorien, con el resto de la comunidad antes de poder formarla y llevarla a Orodruin.

3.8 Complejidad computacional y escalado

La planificación automática clásica es **PSPACE-completa** en su formulación general con acciones proposicionales [1]. Con los requisitos `:typing` y `:conditional-effects` del dominio, el problema permanece en esa clase, pero la fase de *grounding* introduce una complejidad adicional de $O(n^k)$ por acción, donde n es el número de objetos del tipo relevante y k el número de parámetros de la acción.

La acción dominante es `viajarComunidad`, con k parámetros de personaje y $L = 20$ localizaciones. Sin restricciones de tipo, su número bruto de instancias es $|P|^k \cdot L^2$. Los filtros `esDeTipo` y la ruptura de simetrías con `esMenor` lo reducen a:

$$|\text{ground efectivo}| = \binom{H}{h} \cdot \binom{M}{m} \cdot \binom{E}{e} \cdots \times L^2$$

donde cada coeficiente binomial sustituye a un factorial de permutaciones equivalentes. Esto explica por qué el salto de (b) a (c) es casi nulo en tiempo: `esMenor` absorbe las $3! = 6$ permutaciones de Hobbits. El salto de (c) a (d), en cambio, no se debe al *grounding*, sino a la búsqueda en el espacio de estados. Con un planificador heurístico de complejidad $O(b^d)$, el plan de (d) tiene $d = 23$ frente a $d = 17$ en (c), y el factor de ramificación b aumenta al introducir un tipo nuevo (Elfo) con posición inicial en Lothlorien, forzando rutas nuevas para reunirse con el resto. Si b crece un factor α al pasar de (c) a (d), el tiempo escala como $\alpha^{23} \cdot b^{23}$ frente a b^{17} , lo que basta para explicar tres órdenes de magnitud incluso con α moderado.

Una comunidad de 4Ho + 1M + 1E + 1En + 2Hu requeriría que 9 personajes convergieran desde hasta 9 localizaciones distintas. El número de intercalaciones equivalentes del plan de reunificación (del orden de 10^5 – 10^6 para trayectos medios de 3 pasos) sería

$$\frac{(\sum_i d_i)!}{\prod_i d_i!}$$

y harían que el tiempo de planificación excediera cualquier límite práctico sin técnicas adicionales como planificación jerárquica (HTN) o detección explícita de simetrías [2].

3.9 Simetrías residuales

A pesar de las optimizaciones aplicadas, persisten tres fuentes de simetría. La primera es la **simetría de extractor**: si dos Enanos comparten localización, las asignaciones `Enano1 extrae / Enano2 libre` y viceversa son funcionalmente equivalentes, pero `esMenor` no cubre Enanos, por lo que el planificador explora ambas ramas. La segunda es la **simetría de reunificación**: el orden en que se desplazan los k miembros hacia el punto de encuentro no afecta al objetivo. La tercera es la **simetría de Mago**: con dos Magos disponibles (Mago1 en Rivendell, Mago2 en Isengard), la elección de cuál entra en la Comunidad aumenta el espacio de búsqueda sin restricción `esMenor` entre ellos. Estas simetrías no comprometen la completitud, pero sí la eficiencia; en distintas bibliografías se identifican reducciones del espacio de búsqueda de hasta varios órdenes de magnitud al eliminarlas sistemáticamente [2] aunque no se ha creído conveniente para este ejercicio (ya que a excepción del apartado (d) los tiempos son muy bajos y se quería hacer el estudio cambiando solo los parámetros de las acciones, sin modificar apenas el resto).

4. Ejercicio 4

4.1 Diseño del dominio

El Ejercicio 4 extiende el dominio del Ejercicio 3a para modelar la creación de Uruk-Hai en la Torre de Hechicería de Isengard. La jerarquía de tipos incorpora un nuevo tipo raíz: `TipoEdificio`, que representa las categorías de edificio construible (`TorreHechiceria` y `Extractor`).

En las constantes se añaden tres tipos de personaje nuevos (`Orco`, `Humano` y `Corsario`), así como los dos tipos de edificio (`TorreHechiceria` y `Extractor`), declarados directamente como constantes por las mismas razones

de desacople enumeradas en el Ejercicio 1.

Los predicados nuevos incorporados son los siguientes:

- (**edificioEn** [TipoEdificio] [Localizacion]) indica que el edificio del tipo dado ha sido construido en esa localización. Se utiliza para verificar la presencia del **Extractor** antes de extraer Mineral, y la presencia de la **TorreHechiceria** antes de crear el Uruk-Hai.
- (**recursoNecesario** [TipoEdificio] [TipoRecurso]) codifica los requisitos de materiales de cada edificio. Se declara en el problema: la **TorreHechiceria** requiere **Mineral** y **Madera**; el **Extractor** requiere únicamente **Madera**. Este predicado permite que la acción **Construir** itere sobre los recursos necesarios sin recibirlos como parámetro.
- (**urukHaiCreado**) predicado sin parámetros que se activa al ejecutar **CrearUrukHai**. Evita que el Uruk-Hai sea creado más de una vez.
- (**tieneEspecia** [Personaje]) indica que el Corsario porta físicamente Especia consigo. Se activa al ejecutar **RecogerEspecia** y se propaga durante **Viajar**.
- (**especiaEn** [Localizacion]) señala que hay Especia disponible en esa localización. Al viajar, un Corsario con **tieneEspecia** actualiza este predicado: lo retira del origen y lo activa en el destino.
- (**lugarCreacionUruk** [Localizacion]) marca dónde debe crearse el Uruk-Hai (declarado como **Isengard** en el problema).

Las modificaciones sobre las acciones existentes son las siguientes:

- **Viajar** incorpora un efecto condicional: si el personaje porta Especia (**tieneEspecia**), al llegar al destino se activa (**especiaEn ?dst**) y se retira (**especiaEn ?org**), propagando así la Especia sin necesidad de una acción explícita de depósito.
- **ExtraerRecurso** añade una precondition adicional para el caso en que el recurso a extraer sea **Mineral**: debe existir un **Extractor** en la misma localización. Esto modela que la extracción de Mineral requiere infraestructura previa, de acuerdo al enunciado.

Las acciones nuevas introducidas son las siguientes:

Acción	Precondiciones	Efectos
Construir ?constructor ?e ?l	- ?constructor está en ?l y está disponible. - No existe ya un edificio de tipo ?e en ?l. - Si ?e es TorreHechiceria: ?constructor es Mago no miembro. - Si ?e es Extractor: ?constructor es Humano. - Para todo recurso necesario para ?e (forall/exists): existe algún personaje extrayéndolo, y si se trata de la Torre, ese personaje no puede ser Enano.	- El edificio ?e queda construido en ?l. - ?constructor vuelve a estar disponible.
RecogerEspecia ?corsario ?l	- ?corsario es de tipo Corsario, está en ?l y disponible. - El corsario no porta ya Especia. - Hay algún personaje extrayendo Especia en ?l.	- ?corsario pasa a portar Especia (tieneEspecia). - Se activa especiaEn ?l.
CrearUrukHai ?p ?l	- ?p es Mago, no miembro de la Comunidad, está en ?l y disponible. - Existe TorreHechiceria en ?l. - Hay Especia en ?l (especiaEn). - ?l es el lugarCreacionUruk. - El Uruk-Hai no ha sido creado.	- Se activa urukHaiCreado.

Tabla 4: Acciones nuevas/modificadas del dominio 4

4.2 Diseño del problema

Con respecto al Ejercicio 3a se introducen los cambios con respecto al enunciado (no cambia la estructura, solo el contenido en general). En el estado inicial también se declaran los predicados `recursoNecesario` (según las especificaciones del enunciado) y (`lugarCreacionUruk Isengard`).

El objetivo del problema es triple:

- (`anilloDestruido`): el Anillo ha sido destruido en Orodruin por la Comunidad.
- (`urukHaiCreado`): se ha creado un Uruk-Hai en Isengard.
- (`exists (?h - Personaje) (and (esDeTipo ?h Humano) (estaEn ?h Bree))`): algún Humano se encuentra en Bree.

5. Ejercicio 5

5.1 Diseño del dominio

El Ejercicio 5 retoma el dominio del Ejercicio 1 y lo modifica exclusivamente para introducir costes de acción no unitarios, manteniendo intacta la lógica del dominio. La jerarquía de tipos, los predicados y la acción `ExtraerRecurso` son idénticos a los del Ejercicio 1.

El único requisito nuevo es `:action-costs`, que habilita el uso de funciones numéricas y del efecto `increase` en PDDL. Esto sustituye al coste unitario implícito de cada acción por un coste variable declarado en el fichero de problema.

Se añaden dos funciones numéricas al dominio:

- (`costeCamino [Localizacion] [Localizacion]`) almacena el coste de desplazarse entre dos localizaciones concretas. Se inicializa en el fichero de problema para cada par con camino declarado. Los valores siguen las especificaciones del enunciado; los caminos no indicados tienen coste 1.
- (`total-cost`) acumulador estándar de PDDL para el coste total del plan. El planificador minimizará este valor.

La única modificación sobre las acciones existentes recae en `Viajar`:

- Las precondiciones son idénticas al Ejercicio 1.
- Se añade el efecto (`increase (total-cost) (costeCamino ?org ?dst)`), que acumula el coste del desplazamiento en lugar del coste unitario implícito. De este modo el planificador busca la ruta de mínimo coste acumulado, no la de menor número de pasos.

5.2 Diseño del problema

Los objetos (localizaciones, personajes y nodos de recursos), las posiciones iniciales, las capacidades de extracción y el objetivo son idénticos a los del Ejercicio 1. El único cambio en el fichero de problema es la inicialización de la función `costeCamino` para cada par de localizaciones con camino y la inicialización del acumulador (`= (total-cost) 0`). Los costes de los caminos se fijan según el enunciado.

Finalmente, se añade la directiva de optimización (`:metric minimize (total-cost)`), que indica al planificador que debe encontrar el plan de mínimo coste acumulado.

5.3 Análisis y comparación de planes

Para el análisis, recordamos los planes obtenidos en ambos ejercicios. En el Ejercicio 1 el plan encontrado (con coste unitario, longitud 11) es:

```
1 (viajar enano1 tharbad bree)
  (viajar enano1 bree rivendell)
  (viajar hobbit1 lothlorien amonhen)
  (viajar hobbit1 amonhen fangorn)
5 (viajar hobbit1 fangorn isengard)
  (viajar hobbit1 isengard helmsdeep)
  (viajar hobbit1 helmsdeep tharbad)
```

```

(viajar hobbit1 tharbad hobbiton)
(extraerrecurso hobbit1 hobbiton alimento)
10 (viajar enano1 rivendell moria)
(extraerrecurso enano1 moria mithril)
; cost = 11 (unit cost)

```

En el Ejercicio 5 el plan encontrado (con costes variables, coste total 23) es:

```

1 (viajar enano1 tharbad bree)
(viajar enano1 bree rivendell)
(viajar hobbit1 lothlorien amonhen)
(viajar enano1 rivendell moria)
5 (viajar hobbit1 amonhen deadmarshes)
(viajar hobbit1 deadmarshes minasmorgul)
(viajar hobbit1 minasmorgul minastirith)
(viajar hobbit1 minastirith edoras)
(viajar hobbit1 edoras helmsdeep)
10 (viajar hobbit1 helmsdeep tharbad)
(viajar hobbit1 tharbad hobbiton)
(extraerrecurso hobbit1 hobbiton alimento)
(extraerrecurso enano1 moria mithril)
; cost = 23 (general cost)

```

Las diferencias entre ambos planes pueden explicarse a partir de dos factores.

El primero es la **ruta elegida por Hobbit1**. En el Ejercicio 1, el planificador minimiza el número de pasos y elige la ruta directa Lothlorien → AmonHen → Fangorn → Isengard → HelmsDeep → Tharbad → Hobbiton, que tiene 6 pasos (longitud 6). En el Ejercicio 5 esa misma ruta tendría coste $5 + 8 + 1 + 1 + 3 + 1 = 19$. En cambio, la ruta alternativa Lothlorien → AmonHen → DeadMarshes → MinasMorgul → MinasTirith → Edoras → HelmsDeep → Tharbad → Hobbiton tiene 8 pasos (longitud 8) pero coste $5 + 2 + 2 + 2 + 2 + 1 + 3 + 1 = 18$. El planificador escoge esta última por ser de menor coste acumulado, aunque sea más larga en número de acciones. En definitiva, el alto coste del tramo Lothlorien → AmonHen (5) y, sobre todo, el de Fangorn → AmonHen (8) desvía al Hobbit, que aunque añade dos pasos al plan resulta menos costoso en términos métricos.

El segundo factor es la **ruta de Enano1** y el **orden de las acciones**. En ambos ejercicios Enano1 recorre Tharbad → Bree → Rivendell → Moria (3 pasos, coste $1 + 1 + 3 = 5$ en el Ejercicio 5). Sin embargo, en el Ejercicio 1 el planificador secuenciaba el movimiento del Enano solo después de que el Hobbit hubiera comenzado su travesía hacia el oeste, mientras que en el Ejercicio 5 se intercalan las acciones de ambos personajes de forma más entrelazada (puede observarse que **enano1** ya emprende el camino hacia Moria mientras **hobbit1** todavía va en dirección a AmonHen). Este reordenamiento no afecta al coste total, pero sí a la longitud del plan en términos de pasos secuenciales.

En definitiva, la introducción de costes no unitarios cambia el criterio de optimalidad. El plan del Ejercicio 1 es óptimo en longitud (mínimo número de acciones) pero no lo es en coste; el del Ejercicio 5 es óptimo en coste (mínimo coste acumulado) pero tiene mayor longitud. Esto hace ver la diferencia fundamental entre planificación clásica (minimizar longitud) y planificación con métricas de coste (minimizar coste total), y justifica la necesidad del requisito `:action-costs` y de la directiva `:metric` en PDDL cuando los desplazamientos tienen pesos heterogéneos.

Uso de la IA

La IA se ha empleado principalmente para optimizar los programas (todas las optimizaciones implementadas han sido sugeridas por ella aunque solo se han elegido algunas). Además ha ayudado en gran medida al análisis de complejidad del ejercicio 3. Otro uso que se le ha dado ha sido a hacer un esquema inicial de cada ejercicio (añadir las acciones y predicados, en especial su documentación) aunque luego haya sido yo el que las haya implementado (para evitar estar buscando errores y depurando innecesariamente, ya que no tenía mucha complejidad una vez estaban bien planteadas y entendidas).

Referencias

- [1] Bylander, T. (1994). The computational complexity of STRIPS planning. *Artificial Intelligence*, 69(1-2), 165–204.
- [2] Shleyfman, A., Katz, M., Helmert, M., Sievers, S., & Wehrle, M. (2015). Heuristics and symmetries in classical planning. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 29, No. 1, pp. 3371–3377).